

Functions

Function Declaration

Before using the function, the compiler must know about the number and type of parameters that the function expects to receive and the data type of the value that it will return to the calling program.

Placing the function declaration statement prior to its use enables the compiler to make a check on the arguments used while calling that function.

The general format for declaring a function that accepts some arguments and returns some value as a result can be given as :

```
return_data_type function_name(data_type variable 1, data_type variable2,.....)
```

Here, `function_name` is a valid name for function. Naming a function follows the same rules as naming variable.

A function should have a meaningful name that must clarify the task that it will perform.

Following points should be kept in mind about function declaration:

- After the declaration of every function , there should be a semicolon. If the semicolon is missing, the compiler will generate an error message.
- A function cannot be declared within the body of another function.
- A function having void as its return type cannot return any value.
- A function having void as its parameter list cannot accept any value.

Functions Definition

When a function is defined space is allocated for that function in the memory. A function definition comprises two parts:

- Function Header
- Function Body

The Syntax of function definition can be given as:

Return-type function_name(data type variable1, data_type variable2.....)

```
{ .....  
Statements  
.....  
....  
Return variable;  
}
```

Conclusion

A number of conclusions can be drawn regarding functions:

- Any C++ program contains at least one function.
- If a program contains only one function, it must be **main()**.
- In a C++ program if there are more than one function present, the one (and only one) of these functions must be **main()**.
- There is no limit on the number of functions that might be present in a C++ program.
- Each function in a program is called in a sequences specified by the function calls in **main()**.
- After each function has complete its execution control returns to **main()**.

Example Program:

```
#include <iostream>
using namespace std;
int calsum(int x, int y, int z)
{
    int d;
    d = x + y + z;
    return d;
}
int main()
{
    int x, y, z, sum;
    cout << "enter the values for a, b and c:";
    cin >> x >> y >> z;
    sum = calsum(x,y,z);
    cout << " sum = " << sum<<endl;
    system("pause");
    return 0;
}
```

There are a number of things to note about this program:

1. In this program from the main function the values of a, b , c are passed to the function calsum (),by making a call to the function calsum () and mentioning a, b and c in parenthesis.
2. The variables a , b, c are called ‘actual arguments’ , whereas the variables x, y and z are called ‘formal arguments’. Any number of arguments can be passed to a function being called. However, the type, order and number of actual and formal arguments must always be same.

The return statement serves two purposes :

On executing the return statement it immediately transfers the control back to the calling program.

It returns the value present in the parenthesis after return, to the calling program. In the above program value of the sum of three numbers is being returned.

1. There is no restriction on the number of return statements that may be present in a function. Also, the return statement need not always be present at the end of the called function.
2. Whenever control returns from a function some value is definitely returned. If a meaningful value is returned then it should be acceptable in calling program by equating the called function to some variable. For example:

```
Sum = calsum(a,b,c);
```


All the following are valid return statements:

- `return(x);`
- `return (0);`
- `return 1;`
- `return 12.5;`

A function can return only one value at a time . Thus , the following statements are invalid.

`return(a , b);`

`return(x,12);`

If the value of a formal arguments changed in the called function, the corresponding change does not take place in the calling function. For example,

```
#include<iostream>
using namespace std;
void fun(int b)
{
    b = 60;
    cout << b << endl;
}
int main()
{
    int a = 30;
    fun(a);
    cout << a << endl;
    system("pause");
    return 0;
}
```

Thus, even though the value of b is changed in fun(), the value of a in main() remains unchanged . This means that when values are passed to a called function the value present in the actual arguments are not physically moved to the formal arguments, just a photocopy of values in actual argument is made into formal arguments.